

PERFORMANCE TESTING

Performance testing is a type of software testing that focuses on evaluating the performance and scalability of a system or application. Performance testing aims to identify bottlenecks, measure system performance under various loads and conditions, and ensure that the system can handle the expected number of users or transactions.

Performance Testing is the process of analyzing the quality and capability of a product. It is a testing method to determine the system's performance in terms of speed, reliability, and stability under varying workloads.

LOAD TESTING

- The load testing is carried out for normal and extreme load conditions.
- Load testing is a type of performance testing that simulates a real-world load on a system or application to see how it performs under stress.
- The goal of load testing is to identify bottlenecks and determine the maximum number of users or transactions the system can handle.
- It is an important aspect of software testing as it helps ensure that the system can handle the expected usage levels and identify any potential issues before the system is deployed to production.

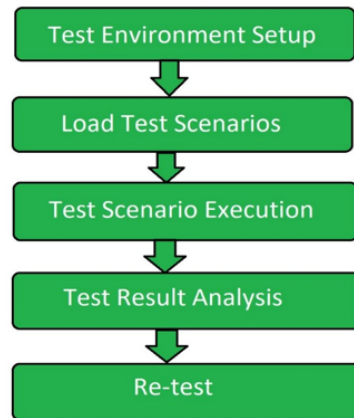
Load Testing Techniques

1. **Stress testing:** Testing the system's ability to handle a high load above normal usage levels
2. **Spike testing:** Testing the system's ability to handle sudden spikes in traffic
3. **Soak testing:** Testing the system's ability to handle a sustained load over a prolonged period of time
4. **Tools for Performance Testing:** Make use of specialized load testing tools like Locust, Gatling, JMeter, LoadRunner, and Apache Benchmark. These tools assist in gathering performance measurements and simulating a large number of users.
5. **Specify the Test Objectives:** Clearly state what your load test's goals are. Recognize the required response times, transaction volumes and expected user behavior.
6. **Determine Crucial Situations:** Determine the essential user scenarios that correspond to common usage patterns. A variety of actions, including user logins, searches, form submissions and other significant interactions, should be covered by these scenarios.

Objectives of Load Testing

1. **Evaluation of Scalability:** Assess the system's ability to handle growing user and transaction demands. Find the point at which the system begins to function badly.
2. **Planning for Capacity:** Describe the system's ability to accommodate anticipated future increases in the number of users, transactions and volume of data. Making well-informed decisions regarding infrastructure upgrades is made easier by this.
3. **Determine bottlenecks:** Identify and localize bottlenecks in the application or infrastructure's performance. Finding the places where the system's performance can suffer under load is part of this.
4. **Analysis of Response Time:** For crucial transactions and user interactions, track and evaluate response times. Make that the system responds to changes in load with reasonable response times.
5. **Finding Memory Leaks:** Find and fix memory leaks that may eventually cause a decline in performance. Make sure the programmer doesn't use up too many resources when it's running.

Load Testing Process



1. **Test Environment Setup:** Firstly create a dedicated test environment setup for performing the load testing. It ensures that testing would be done in a proper way.
2. **Load Test Scenario:** In second step load test scenarios are created. Then load testing transactions are determined for an application and data is prepared for each transaction.
3. **Test Scenario Execution:** Load test scenarios that were created in previous step are now executed. Different measurements and metrics are gathered to collect the information.
4. **Test Result Analysis:** Results of the testing performed is analyzed and various recommendations are made.
5. **Re-test:** If the test is failed then the test is performed again in order to get the result in correct way.

Metrics of Load Testing

Metrics are used in knowing the performance of load testing under different circumstances. It tells how accurately the load testing is working under different test cases. It is usually carried out after the preparation of load test scripts/cases.

1. Average Response Time

It tells the average time taken to respond to the request generated by the clients or customers or users. It also shows the speed of the application depending upon the time taken to respond to the all requests generated.

2. Error Rate

The Error Rate is mentioned in terms of percentage denotes the number of errors occurred during the requests to the total number of requests. These errors are usually raised when the application is no longer handling the request at the given time or for some other technical problems. It makes the application less efficient when the error rate keeps on increasing.

3. Throughput

This metric is used in knowing the range of bandwidth consumed during the load scripts or tests and it is also used in knowing the amount of data which is being used for checking the request that flows between the user server and application main server. It is measured in kilobytes per second.

STRESS TESTING

Stress testing is defined as types of software testing that verifies the stability and reliability of the system. This test particularly determines the system's robustness and error handling under the burden of some load conditions. It tests beyond the normal operating point and analyses how the system works under extreme conditions. Stress testing is performed to ensure that the system does not crash under crunch situations. Stress testing is also known as Endurance Testing.

Characteristics of Stress Testing

1. **Identification of Risk:** Stress testing's main objective is to locate and evaluate a system's possible hazards and weaknesses.
2. **Quantitative and Qualitative Analysis:** While numerical data are crucial, it's also critical to comprehend the qualitative characteristics of the system's response and potential weak points.
3. **Variable Parameters:** Stress testing includes changing variables including interest rates, market conditions, transaction volumes and outside influences that could have an impact on the system.

4. **Cross-Functional Involvement:** Many departments within an organization must work together and participate in stress testing. This cross-functional strategy makes sure that the stress testing procedure benefits from a variety of viewpoints and specialties.
5. **Open and Honest Communication:** Stress testing necessitates open and honest communication regarding the goal, approach, and outcomes of the testing procedure.

Purpose of Stress Testing

- **Analyze the behavior of the application after failure:** The purpose of stress testing is to analyze the behavior of the application after failure and the software should display the appropriate error messages while it is under extreme conditions.
- **System recovers after failure:** Stress testing aims to make sure that there are plans for recovering the system to the working state so that the system recovers after failure.
- **Uncover Hardware issues:** Stress testing helps to uncover hardware issues and data corruption issues.
- **Uncover Security Weakness:** Stress testing helps to uncover the security vulnerabilities that may enter into the system during the constant peak load and compromise the system.
- **Ensures data integrity:** Stress testing helps to determine the application's data integrity throughout the extreme load, which means that the data should be in a dependable state even after a failure.

Stress Testing Process

The stress testing process is divided into 5 steps:

1. Planning the stress test

- **Gathering System Data:** For a web application, collect details about server specifications, database configurations, network bandwidth, and current traffic patterns.
- **Analyzing the System:** Evaluate how the web application performs during peak traffic times, such as during a sales event. Identify any existing issues like slow page loads or database timeouts.
- **Defining Stress Test Goals:** Set goals such as ensuring the application can handle a traffic spike of 10,000 users per minute without crashing and maintaining response times below 2 seconds.

2. Create Automation Scripts

- **Developing Stress Testing Scripts:** Use tools like Apache JMeter or LoadRunner to create scripts that simulate user actions such as logging in, browsing products, and making purchases.
- **Generating Test Data:** Create datasets that include various user profiles, product inventories, and transaction records to ensure the test is realistic and comprehensive.

3. Script Execution

- **Analyzing Test Data:** Review the logs to identify performance trends. Look for any significant slowdowns or failures as the load increases.
- **Identifying Bottlenecks:** If the application slows down significantly at 5,000 users, investigate the server logs to determine if the issue is related to database queries, server CPU limits, or network bandwidth.

4. Result Analysis

This phase involves analyzing stress test results and identifying the bottlenecks.

- **Running the Stress Test Scripts:** Execute the scripts in a test environment that mirrors the production setup. Gradually increase the number of simulated users to observe how the system handles the load.
- **Storing Test Results:** Use logging tools to capture performance data such as server response times, error rates, and resource utilization metrics.

5. Tweaking and Optimization

This step involves fine-tuning the system and optimizing the code with the goal to meet the desired benchmarks.

- **Fine-Tuning the System:** Based on the findings, optimize database queries, increase server CPU capacity, or enhance network configurations.
- **Optimizing for Desired Benchmarks:** Retest the application to ensure it now handles 10,000 users per minute with response times below 2 seconds.

VOLUME TESTING

Volume Testing is a type of software testing which is carried out to test a software application with a certain amount of data. The amount used in volume testing could be a database size or it could also be the size of an interface file that is the subject of volume testing. While testing the application with a specific database size, database is extended to that size and after that the performance of the application is tested.

Characteristics of Volume Testing

- Performance of the software decline as passing of the time as there is huge amount of data overtime.
- Basically the test data is created by test data generator.
- Only small amount of data is tested during development phase.
- The test data need to be logically correct.
- The test data is used to assess the performance of the system.

Objectives of Volume Testing

- To recognize the problems that may be created with large amount of data.
- To check the system's performance by increasing the volume of data in the database.
- To find the point at which the stability of the system reduces.
- To identify the capacity of the system or application.

•

Volume Testing Attributes

- **System's Response Time:** During the volume testing, the response time of the system or the application is tested. It is also tested whether the system responses within the finite time or not. If the response time is large then the system is redesigned.
- **Data Loss:** During the volume testing, it is also tested that there is no data loss. If there is data loss some key information might be missing.
- **Data Storage:** During the volume testing, it is also tested that the data is stored correctly or not. If the data is not stored correctly then it is restored accordingly in proper place.
- **Data Overwriting:** In volume testing, it is tested that whether the data is overwritten without giving prior information to the developer. If it so then developer is notified.

Advantages of Volume Testing

- Volume testing is helpful in saving maintenance cost that will be spent on application maintenance.
- Volume testing is also helpful in a rapid start for scalability plans.
- Volume testing also helps in early identification of bottlenecks.
- Volume testing ensures that the system is capable of real world usage.

FAIL-OVER TESTING

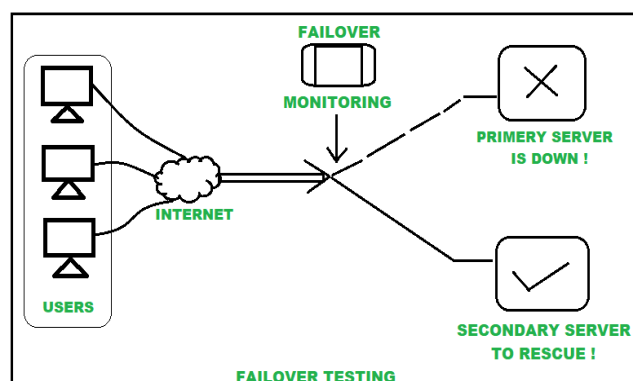
Software products are tested multiple times before they are released to ensure they work as expected. However, testing before delivery doesn't guarantee that issues won't arise in the future. Sometimes, software can fail due to unexpected events like network problems or server failures. Failover testing is designed to check how well the software can recover from such failures.

For example, imagine your computer shuts down unexpectedly. When you restart your browser, a pop-up asks if you want to restore all the pages you had open. Clicking "Restore" brings back all the tabs exactly as they were. This process of recovering and restoring the previous state of the application is what failover testing ensures.

There are two common configurations for failover testing: **active-active** and **active-passive standby**.

- **Active-Active:** In this setup, all servers are running and share the load equally.
- **Active-Passive Standby:** Here, one server takes the main load, and the backup server remains idle until the primary one fails.

Working of Failover testing



1. **Consider the factors:** Before performing failover testing like budget, time, team, technology, etc.
2. **Analysis on failover reasons and design solutions:** Determine probable failure situations that the system might experience. Examine the causes of failover, including software bugs, hardware malfunctions, network problems, etc. It provides fixes for any flaws or vulnerabilities found in the failover procedure.
3. **Testing failover scenarios:** It develops extensive test cases to replicate various failover scenarios. This covers both unplanned failovers (system or component failures) and scheduled failovers (maintenance). Test cases ought to address many facets of failover, such as load balancing, user impact, network rerouting, and data synchronization.
4. **Executing the test plan:** To reduce the impact on production systems, carry out the failover test plan in a controlled setting. Keep an eye on how the system behaves during failover to make sure it satisfies the recovery point and recovery time objectives (RPO and RTO, respectively).
5. **Detailed report on failover:** Keep a record of the failover testing findings, including any problems you ran across, how long it took to failover and how it affected customers or services. Assess problems according to their severity and offer suggestions for improvements.
6. **Necessary actions based on the report:** Distribute the report on the failover test to all pertinent parties, such as project managers, developers, and system administrators.

Benefits of Failover Testing

1. **Determines Vulnerabilities and Weaknesses:** Failover testing helps in locating holes and vulnerabilities in the system by simulating different failure scenarios.
2. **Verifies Redundancy Procedures:** Redundancy mechanisms are frequently incorporated into systems to improve reliability. In order to make sure that these systems such as redundant servers, databases, or network paths, work flawlessly in the event of a breakdown, failover testing verifies their efficacy.
3. **Improving the User Experience:** Consumers anticipate consistent and dependable service availability. By reducing interruptions and ensuring service availability even in the face of unanticipated circumstances, failover testing contributes to the provision of a positive user experience.
4. **Encourages Compliance:** Failover testing supports organizations in demonstrating compliance with these rules by ensuring that systems can recover within prescribed times.
5. **Encourages Continuous Improvement:** Organizations that regularly test their systems can learn from each test cycle, fix problems found and upgrade their failover procedures to meet changing business needs and technological advancements.

RECOVERY TESTING

Recovery Testing is a crucial aspect of software testing focused on ensuring that applications can recover quickly and effectively from crashes, failures, and unexpected interruptions. This type of testing verifies the software's robustness and its ability to restore operations after a malfunction. Recovery testing is also referred to as Disaster Recovery Testing.

Recovery tests for failures

A system or software should be recovery tested for failures like:

- Power supply failure
- The external server is unreachable
- Wireless network signal loss
- Physical conditions
- The external device not responding
- The external device is not responding as expected, etc.

Steps to be performed before executing a Recovery Test

1. **Recovery Analysis** – It is important to analyze the system's ability to allocate extra resources like servers or additional CPUs. This would help to better understand the recovery-related changes that can impact the working of the system. Also, each of the possible failures, their possible impact, their severity, and how to perform them should be studied.
2. **Test Plan preparation** – Designing the test cases keeping in mind the environment and results obtained in recovery analysis.
3. **Test environment preparation** – Designing the test environment according to the recovery analysis results.
4. **Maintaining Back-up** – Information related to the software, like various states of the software and database should be backed up. Also, if the data is important, then the backing up of the data at multiple locations is important.
5. **Recovery personnel Allocation** – For the recovery testing process, it is important to allocate recovery personnel who are aware and educated enough for the recovery testing being

conducted.

6. **Documentation** – This step emphasizes on documenting all the steps performed before and during the recovery testing so that the system can be analyzed for its performance in case of a failure.

Example:

When a browser is working on multiple sessions, we can stimulate software failure by restarting the system. After restarting the system, we can check if it recovers from the failure and reloads all the sessions it was previously working on.

Advantages:

- Improves the quality of the system
- Risk elimination
- Improved performance
- Ensures Reliability
- Identifies Weaknesses
- Enhances User Experience
- Improves System Stability
- Supports Business Continuity
- Validates Backup and Recovery Procedures
- Reduces Costs
- Boosts Confidence

CONFIGURATION TESTING

Configuration Testing is the process of testing the system under each configuration of the supported software and hardware. Here, the different configurations of hardware and software mean the multiple operating system versions, various browsers, various supported drivers, distinct memory sizes, different hard drive types, various types of CPU, etc.

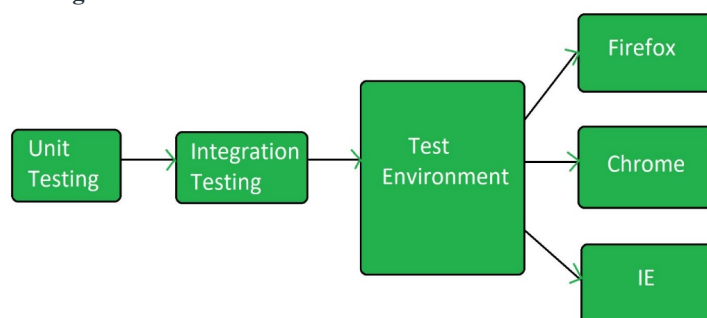
The various configurations are Win XP, Win 7 32/64 bit, Win 8 32/64 bit, Win 10, etc.

1. **Database Configuration:** Oracle, DB2, MySQL, MSSQL Server, Sybase etc.
2. **Browser Configuration:** IE 8, IE 9, FF 16.0, Chrome, Microsoft Edge etc.

Objectives of Configuration Testing:

1. **Adaptability to Different Configurations:** Check that the program's basic features work consistently and dependably in all configurations. Testing the behavior of the program with different setups and settings is part of this process.
2. **Evaluation of Stability:** Examine the software's stability under various configurations. Find and fix any configuration-specific problems that might be causing crashes, unstable systems or strange behavior.
3. **Testing the User Experience:** Assess the value and consistency of the user experience across various setups. Make that the graphical user interface (GUI) of the software adjusts to various screen sizes, resolutions and display settings.
4. **Security throughout Configurations:** To make sure that sensitive data is kept safe, test the software's security features in various setups. Determine and fix any vulnerabilities that might be configuration-specific.
5. **Compatibility of Networks:** Examine the software's behavior with various network setups. Evaluate its compatibility with various network types, speeds and latency.
6. **Data Compatibility:** Check if the programme can manage a range of data configurations, such as those from diverse sources, databases and file formats. Verify the consistency and integrity of the data across various setups.

Configuration Testing Process:



Types of Configuration Testing:

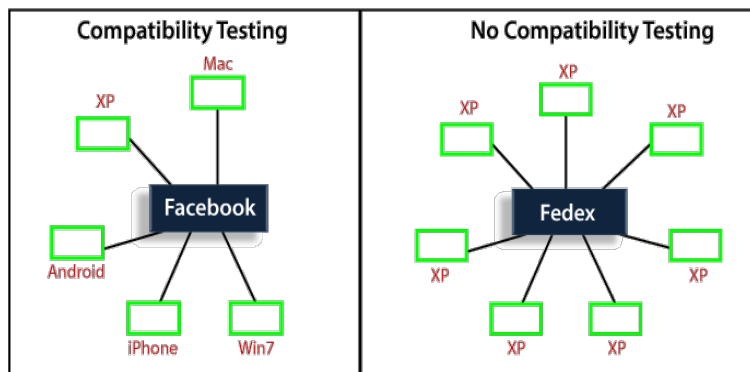
1. **Software Configuration Testing:** Software configuration testing is done over the Application

Under Test with various operating system versions and various browser versions etc. It is a time-consuming testing as it takes long time to install and uninstall the various software which are to be used for testing. When the build is released, software configuration begins after passing through the unit test and integration test.

2. **Hardware Configuration Testing:** Hardware configuration testing is typically performed in labs where physical machines are used with various hardware connected to them. When a build is released, the software is installed in all the physical machines to which the hardware is attached and the test is carried out on each and every machine to confirm that the application is working fine. While doing hardware configuration test, the kind of hardware to be tested is spelled out and there are several computer hardware and peripherals which make it next to impossible to execute all the tests.

COMPATIBILITY TESTING

Checking the functionality of an application on different software, hardware platforms, network, and browsers is known as compatibility testing. Once the application is stable, we moved it to the production, it may be used or accessed by multiple users on the different platforms, and they may face some compatibility issues, to avoid these issues, we do one round of compatibility testing.



Types of Compatibility testing

1. Software
2. Hardware
3. Network
4. Mobile

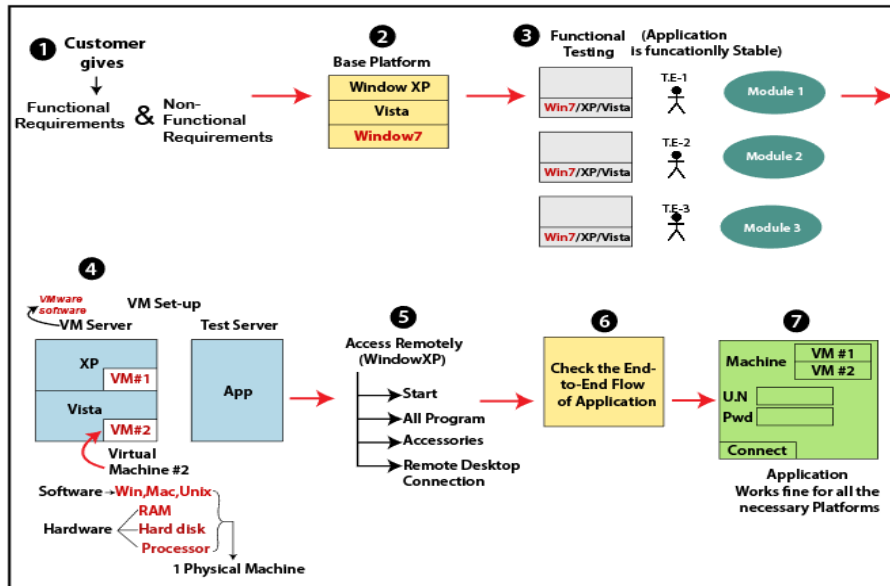
Software- Software means different operating systems (Linux, Window, and Mac) and also check the software compatibility on the various versions of the operating systems like Win98, Window 7, Window 10, Vista, Window XP, Window 8, UNIX, Ubuntu, and Mac.

Hardware - The application is compatible with different sizes such as RAM, hard disk, processor, and the graphic card, etc.

Mobile - Check that the application is compatible with mobile platforms such as iOS, Android, etc.

Network - Checking the compatibility of the software in the different network parameters such as operating speed, bandwidth, and capacity.

Compatibility testing process



1. Firstly, the customer will give the functional requirement as well as a non-functional requirement.
2. After getting the non-functional requirement, the one base platform will decide according to the most commonly used platform.
3. After that, the test engineer will start functional testing on the base platform until the application is functional stable.
4. We have to test the application on a different platform, so for this, we have a VMware software.
5. For compatibility testing, we will use VM Server where we install all necessary operating system and browser, and access that server using Remote Desktop connection.
6. We access VMware set up remotely, and after that, the test engineer will do one round of compatibility testing on the platform and check the end-to-end flow.
7. The end-to-end flow will go on until the application is stable, and applications work fine for all necessary platforms and handover to the customer.

USABILITY TESTING

Usability Testing is a significant type of software testing technique, which comes under the [non-functional testing](#). It is primarily used in user-centered interaction design on order to check the usability or ease of using a software product. The implementation of usability testing requires an understanding of the application, as it is extensive testing.

Characteristics:

1. Easy to understand
2. Easy to access
3. Look and feel
4. Faster to Access
5. Effective Navigation
6. Good Error Handling

Features

- It is an essential type of non-functional testing technique, which comes under the black-box testing technique in software testing.
- Usability testing is performed throughout the system and acceptance testing levels.
- Generally, usability testing is implemented in the early stage of the Software Development Life Cycle (SDLC).
- The execution of usability testing offers more visibility on the prospects of the end-users.
- The usability testing makes sure that the software product meets its planned purpose.
- It also helps us to find many usability errors in the specified software product.
- Usability testing mainly tests the user-friendliness, usefulness, traceability, usability, and desirability of the final product.
- It offers direct input on how real-users use the software/application.
- The usability testing includes the systematic executions of the product's usability under a measured environment.

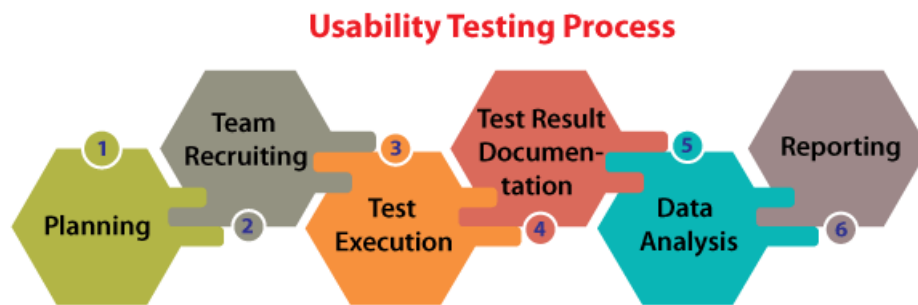
Usability Testing Process

The Process of usability testing is completed into few significant steps. This process will assist us in providing and creating different results for all the issues identified during the execution of testing.

In real-time, usability testing tests the application's behavior from the user's perspective even though it is a time-consuming process, providing the tester the most precise outcomes from actual testing.

And that gives us an idea of the errors/flaws in our product and helps us distinctly before installing it on the server.

The usability testing process follows a precise set of steps to help the team get the detailed and helpful response from the end-users.



Step1: Planning

The first step of usability testing is planning, where the team makes the test plan and generates some document samples that help the testing team complete the usability testing tasks.

The objective of the usability test is governed in the planning step. Here, the aim is not to have the volunteers sit in front of our application and recording their activities, but we need to fix the crucial features and elements of the system.

We need to give the tasks to our test engineers who are familiar with these crucial features. And the usability testing method, number, and demographics of usability test engineers, test report formats are also fixed throughout the planning phase.

Step2: Team Recruiting

Once the planning phase is completed, we will move to the next step of usability testing, which is **team recruiting**.

As the name suggests itself, here, we will hire or recruit the end-user delegates and the participants or the test engineers as per the budget and density of the product.

These representatives or test engineers are prepared to sit across the test sessions and validate the correctness and usability of the product.

Primarily, the selection of these test engineers is based on the necessity of testing along with the number of persons mentioned in the test plan.

As soon as the hiring of a test engineer is achieved, the team is appointed to the particular responsibilities and jobs.

Step3: Test Execution

Once the planning and team recruiting steps have been completed successfully. We will proceed to the next step, which is **Test Execution**.

In the test execution step, the test engineers execute the usability testing and implement their assigned responsibilities. In this situation, the users' needs to test the product to find irregularity, if any, and also record them correctly.

Step4: Test Result Documentation

The **test result documentation** step includes the results based on the **test execution** step and then proceeds for further analysis.

Step5: Data Analysis

Once the test result documentation is done, we will move to the next step of the usability testing process, i.e., **Data Analysis**.

The response or the feedback is obtained from usability testing evaluation in the data analysis phase. And the outcomes are classified, and the patterns are acknowledged.

In this step, the data from usability tests are wholly evaluated to get expressive implications and help us provide actionable suggestions to enhance the overall usability of our product.

Step6: Reporting

After performing all the above steps successfully, we will finally reach the last step of the usability testing process which is named as **Reporting**.

Example

We are taking one **banking application** where we produce the application for the manager.

Now, if the end-user (manager) starts using the application in front of the test engineers

Suppose two test engineer sits at the back of the end-user while he/she is using the application and takes the report of the defect as a developer to check whether the end-user is using the application in a right way or not.

And the end-user (manager) will check the application step by step because he/she knows that the Test engineer is watching him/her.

Sometimes Test engineer has to do usability testing for the following reasons:

- There is no money to spend on usability testing.
- Do not want to outsource to another company.

SECURITY TESTING

Security Testing is a type of Software Testing that uncovers vulnerabilities in the system and determines that the data and resources of the system are protected from possible intruders. It ensures that the software system and application are free from any threats or risks that can cause a loss. Security testing of any system is focused on finding all possible loopholes and weaknesses of the system that might result in the loss of information or reputе of the organization.

Types of Security Testing

1. Vulnerability Scanning

It is a type of testing that uses automated tools to scan the system for known vulnerabilities and weaknesses. It aims to detect patterns of vulnerabilities that are commonly exploited by attackers. By performing vulnerability scans regularly, organizations can proactively address these vulnerabilities before they become security risks.

2. Security Scanning

It involves identifying weaknesses in the network or system and then providing solutions to mitigate these risks. It can be performed either manually or automatically, depending on the complexity of the system. This process helps uncover potential weak points that could be exploited by attackers, allowing for early intervention to secure the system.

3. Penetration Testing

It simulates an attack from a malicious hacker to identify vulnerabilities in the system. This type of testing helps organizations understand how an attacker might exploit weaknesses in the system. By performing penetration testing, organizations can see their system from an attacker's perspective and fix vulnerabilities before they are exploited in a real-world attack.

4. Risk Assessment

It involves analyzing the security risks that could affect the organization. Risks are categorized as low, medium, or high, and this testing suggests controls and measures to minimize those risks. Risk assessment helps prioritize actions by identifying the most critical threats and focusing efforts on addressing them first, ultimately improving the overall security posture of the system.

5. Security Auditing

These is an internal inspection of the system to identify security defects. This can involve reviewing system configurations, checking for weaknesses in the code, or conducting a line-by-line inspection of the application's source code. Security audits ensure that all security standards and protocols are being followed and identify any gaps that need to be addressed.

6. Ethical Hacking

It also known as white-hat hacking, is when security professionals are hired to simulate attacks on the system to identify vulnerabilities. Unlike malicious hacking, ethical hacking is done with the organization's consent to help improve system security. Ethical hackers use the same techniques as malicious hackers to uncover weaknesses in the system, but their goal is to fix those flaws before they can be exploited by actual attackers.

7. Posture Assessment

These will combines security scanning, ethical hacking, and risk assessments to provide an overall view of the system's security. It gives a comprehensive evaluation of the system's security by integrating multiple testing methods, ensuring that no part of the security infrastructure is overlooked. This assessment helps organizations understand their security readiness and take necessary actions to strengthen their defenses.

8. Application Security Testing

These testing focuses specifically on identifying vulnerabilities within the application itself. This includes examining the application's code, configurations, and dependencies to identify flaws that could lead to security breaches. Regular application security testing ensures that the software does not contain any weaknesses that could be exploited by attackers.

9. Network Security Testing

In these testing targets the vulnerabilities in the network infrastructure, such as firewalls, routers,

and other network devices. This testing is crucial for identifying weaknesses that could allow unauthorized access to the system. Network security testing helps ensure that the communication pathways between devices are secure and that sensitive data is protected from cyber threats.

10. Social Engineering Testing

Social Engineering Testing simulates phishing, baiting, or other manipulative techniques used to exploit human behavior to gain unauthorized access. This type of testing focuses on the human element of security, ensuring that employees are aware of potential threats and know how to protect themselves from such attacks.

Goal of Security Testing

- To identify the threats in the system and measure the potential vulnerabilities of the system.
- To help in detecting every possible security risk in the system and help developers fix security problems through coding.
- The goal of security testing is to identify vulnerabilities and potential threats in a system or application and to ensure that the system is protected against unauthorized access, data breaches, and other security-related issues.
- Identify vulnerabilities: Security testing helps identify vulnerabilities in the system, such as weak passwords, unpatched software, and misconfigured systems that could be exploited by attackers.
- Evaluate the system's ability to withstand an attack: Security testing evaluates the system's ability to withstand different types of attacks, such as network attacks, social engineering attacks, and application-level attacks.
- Ensure compliance: Security testing helps ensure that the system meets relevant security standards and regulations, such as HIPAA, PCI DSS, and SOC2.
- Provide a comprehensive security assessment: Security testing provides a comprehensive assessment of the system's security posture, including the identification of vulnerabilities, the evaluation of the system's ability to withstand an attack, and compliance with relevant security standards.
- Help organizations prepare for potential security incidents: Security testing helps organizations understand the potential risks and vulnerabilities that they face, enabling them to prepare for and respond to potential security incidents.
- Identify and fix potential security issues before deployment to production: Security testing helps identify and fix security issues before the system is deployed to production. This helps reduce the risk of a security incident occurring in a production environment.

Principle of Security Testing

1. **Confidentiality:** Verifies that sensitive data is only accessible to authorized users, often through encryption and access control mechanisms.
2. **Integrity:** Verifies that data remains unchanged and unaltered during storage or transmission. Hash functions and checksums are commonly used to guarantee integrity.
3. **Authentication:** Ensures that only authorized users can access the system. This involves testing password policies, multi-factor authentication (MFA), and identity verification mechanisms.
4. **Authorization:** Verifies that authenticated users can only access the resources and data they are authorized to use, through mechanisms such as role-based or attribute-based access control (RBAC and ABAC).
5. **Availability:** Ensures that the system remains functional and accessible, even under heavy traffic or during a cyberattack, such as a Distributed Denial of Service (DDoS) attack.
6. **Non-Repudiation:** Ensures that users cannot deny their actions in the system. Digital signatures, audit logs, and transaction records are commonly used to guarantee non-repudiation.
7. **Resilience:** Verifies the system's ability to recover from incidents, such as system crashes or attacks, by evaluating backup systems and response protocols.

Advantages

- Identifying vulnerabilities
- Improving system security
- Ensuring compliance
- Reducing risk
- Improving incident response

TESTING DOCUMENTATION

Testing documents are prepared at different stages. These documents are discussed as follows.

1. Before Testing:

- **SRS document** – Functional Requirements document.
- **Test Policy document** – It means the product must be tested far before release.
- **Test Strategy document** – It mentions detailed aspects of test the team, responsibility matrix, and rights/responsibilities of the test manager and test engineer.
- **Traceability Matrix document** – This is [SDLC document](#), that is related to the requirements-gathering process. As new requirements come, they are added to this matrix. They can be traced forward and backward. These matrices help testers know the source of the requirement.

2. During Testing:

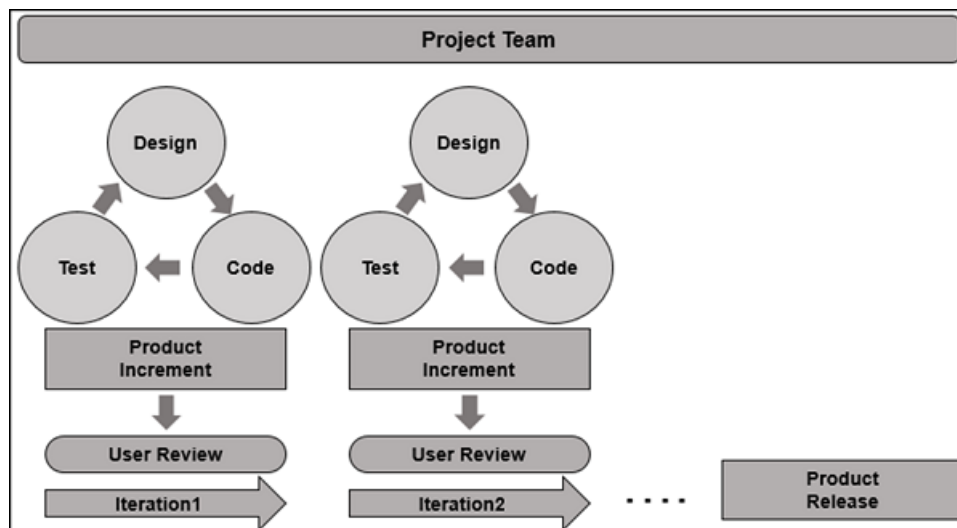
- **Test Case document** – It contains the list of to-be tests. It includes various testing like Unit test plan, [Integration test plan](#), [System test plan](#) and [Acceptance test plan](#).
- **Test description** – It is a detailed description of all test cases and procedures for executing them.
- **Test case report** – It contains a test case report resulting from the test.
- **Test logs** – It contains test logs for every test case report.

3. After Testing:

After testing, only the test summary remains which is a collective analysis of all test reports and logs. The software is released under the version control system if it is ready to launch. It summarizes and concludes whether the software is ready to launch.

TESTING IN THE AGILE ENVIRONMENT

- Agile is an iterative development methodology, where the entire project team participates in all the activities.
- The requirements evolve as the iterations progress, through collaboration between the customer and the self-organizing teams.
- As Coding and Testing are done interactively and incrementally, during the course of development, the end-product would be of quality and ensures customer requirements.
- Every iteration results in an integrated working product increment and is delivered for User Acceptance Testing.
- The customer feedback thus obtained would be an input to the next / subsequent Iterations.



Continuous Integration, Continuous Quality

- Continuous Integration is the key for Agile Development success. Integrate frequently, at least daily such that you are ready for a release as and when required.
- Testing in Agile becomes an essential component of all the phases of the development, ensuring continuous quality of the product.
- Constant feedback from everyone involved in the project adds to the quality of the product.

- In Agile, communication is given utmost importance and the customer requests are received as and when necessary.
- This gives the satisfaction to the customer that all the inputs are considered and working quality product is available throughout the development.

Agile Testing Methodologies

1. The testing practices are well defined for every project, whether Agile or not, to deliver quality products.
2. Traditional Testing principles are quite often used in Agile Testing. One of them is Early Testing that focuses on –
 - Writing Test Cases to express the behavior of the system.
 - Early Defect Prevention, detection and removal.
 - Ensuring that the right test types are run at the right time and as part of the right test level.

The other commonly used Agile Testing Methodologies are –

- Test-Driven Development (TDD) – Test-Driven Development (TDD) is based on coding guided by tests.
- Acceptance Test-Driven Development (ATDD) – Acceptance Test-Driven Development (ATDD) is based on communication between the customers, developers and testers and driven by pre-defined Acceptance Criteria and Acceptance Test Cases.
- Behavior-Driven Development (BDD) – In Behavior-Driven Development (BDD) testing is based on the expected behavior of the software being developed.

TESTING THE WEB AND MOBILE APPLICATIONS:

What is a Web Application?

A web app is an application that runs on a web browser and is accessed over the internet, without the need to download it.

Example: Gmail, Google Docs

What is a Mobile Application?

A mobile app is an application designed for smartphones and tablets, usually available for download through app stores.

Example: WhatsApp, Uber

Types of Web Applications

- Static Web Applications
- Dynamic Web Applications
- E-Commerce Applications
- Single-Page Applications (SPAs)
- Progressive Web Applications (PWAs)
- Content Management Systems (CMS)
- Enterprise Web Applications
- Portal Web Applications

1. Static Web Applications

Simple web applications that primarily display fixed content and do not interact dynamically with users.

Technology Stack: HTML, CSS, and JavaScript (basic level).

Features:

- Easy to build and deploy.
- Limited interactivity.

Examples: Personal blogs, portfolio websites.

2. Dynamic Web Applications

Applications that generate content dynamically based on user interactions and data.

Technology Stack: Backend languages (for example, PHP, Python, Ruby), frameworks, and databases.

Features:

- Real-time data updates.
- Requires a backend for database interaction.

Examples: Social media platforms, e-commerce sites.

3. E-Commerce Applications

Web applications specifically designed for online shopping and transactions.

Technology Stack: Backend (for example, Node.js, Ruby on Rails), frontend frameworks (React, Angular), and payment gateways.

Features:

- Product catalogs, shopping carts, payment integration.
- Secure and scalable design.

Examples: Amazon, eBay, Shopify.

4. Single-Page Applications (SPAs)

Applications that load a single HTML page and dynamically update content without reloading the page.

Technology Stack: Frameworks like React, Angular, Vue.js.

Features:

- Fast and seamless user experience.
- Heavy reliance on JavaScript.

Examples: Gmail, Google Maps, Trello.

5. Progressive Web Applications (PWAs)

Web applications that behave like native mobile apps while being accessible through browsers.

Technology Stack: HTML, CSS, JavaScript, Service Workers.

Features:

- Offline capabilities.
- Cross-platform compatibility.
- Installation as an app-like icon on devices.

Examples: Twitter Lite, Starbucks PWA.

6. Content Management Systems (CMS)

Applications that allow users to create, edit, and manage digital content without coding knowledge.

Technology Stack: WordPress, Joomla, Drupal.

Features:

- User-friendly interfaces.
- Customizable templates.

Examples: News websites, blogs, business websites.

7. Enterprise Web Applications

Applications built to meet the needs of organizations for business processes, communication, and productivity.

Technology Stack: Backend (Java, .NET), frontend (React, Angular), and robust databases.

Features:

- Scalable and secure.
- Integration with enterprise systems.

Examples: ERP systems, CRM platforms like Salesforce.

8. Portal Web Applications

Applications that provide a gateway to access diverse services, content, or information.

Technology Stack: Backend languages and frameworks for user authentication and data management.

Features:

- Role-based access.
- Customizable dashboards.

Examples: Student portals, employee portals.

Types of Mobile Applications

1. Native Applications

These apps are built specifically for a mobile operating system (for example, Android or iOS) using platform-specific programming languages.

Technology Stack:

- **Android:** Java, Kotlin
- **iOS:** Swift, Objective-C

Advantages:

- High performance and responsiveness.
- Full access to device features (for example, camera, GPS).
- Better user experience with platform-specific UI/UX.

Examples: WhatsApp, Google Maps, and Facebook (native versions).

2. Mobile Web Applications

These apps run on mobile browsers and are not installed on devices. They use responsive design to function across different screen sizes.

Technology Stack: HTML, CSS, JavaScript, and frameworks like ReactJS, AngularJS.

Advantages:

- Cross-platform compatibility (accessible via browsers).
- No installation is required, reducing storage usage.
- Easier to update and maintain.

Examples: Twitter Lite, Google Search on mobile browsers.

3. Hybrid Applications

Hybrid apps combine elements of both native and web applications, typically wrapped in a native shell to allow access to device features.

Technology Stack: Frameworks like React Native, Flutter, Ionic, Cordova.

Advantages:

- Cross-platform development with a single codebase.
- Faster development compared to native apps.
- Can access device features using plugins.

Examples: Instagram, Uber, Evernote.

Tools or Frameworks for Web Application Testing

- ✓ BrowserStack
- ✓ Selenium
- ✓ Jira
- ✓ Cypress
- ✓ Postman
- ✓ Playwright
- ✓ Tools or Frameworks Used for Mobile App Testing
- ✓ BrowserStack
- ✓ Appium
- ✓ Espresso
- ✓ XCUITest
- ✓ Calabash

Difference between Web and Mobile Applications

Aspect	Web Applications	Mobile Applications
Platform Dependency	Runs on web browsers; platform-independent.	Platform-specific (Native) or cross-platform (Hybrid).
Installation	No installation required; accessed via URL.	Requires installation from app stores (for example, Google Play, App Store).
Internet Dependency	Requires an internet connection unless cached (for example, PWAs).	Can work offline (for native apps) depending on the design.
Development	Built using web	Developed using platform-specific

	technologies (HTML, CSS, JavaScript, etc.).	languages (Swift, Kotlin) or frameworks like Flutter.
User Experience (UX)	Limited by browser capabilities; relies on responsive design.	Richer and more intuitive, utilizing device features like GPS and cameras.
Performance	Dependent on browser and internet speed.	Generally faster, with direct access to hardware resources.
Updates	Instantly updated on the server side.	Requires users to download updates from the app store.
Device Features	Limited access to device features (some via browser APIs).	Full access to device hardware like cameras, accelerometers, etc.
Offline Capability	Limited; mostly requires a stable internet connection.	Better offline support (for example, native apps store data locally).
Cost	Cheaper to develop and maintain as it works across platforms.	More expensive, especially for separate native apps for each OS.
Examples	Google Docs (web version), Amazon (website).	WhatsApp, Instagram, Uber.

Tools or Frameworks for Web Application Testing

Selenium

Selenium is a widely used open-source tool for automating web browser testing. It supports multiple browsers, platforms, and programming languages.

Key Features:

- Supports cross-browser testing on Chrome, Firefox, Safari, etc.
- Compatible with multiple programming languages like Java, Python, and C#.
- Provides Selenium Grid for parallel test execution.
- Flexible integration with CI/CD tools like Jenkins.

Pros:

- Highly customizable and extendable.
- Large community support and extensive documentation.
- Open-source with no licensing cost.

Cons:

- Requires advanced programming skills.
- Limited support for modern web app features like shadow DOM.
- Initial setup can be complex

What is Selenium?

Selenium is an open-source framework designed for automating web browsers. It enables users to test website functionality across different browsers, ensuring consistency and compatibility.

Key features of Selenium:

- Supports functional and cross-browser testing to verify seamless performance across

multiple browsers.

- Allows test script creation in various programming languages, including Java, Python, JavaScript, Ruby, C#, and more.
- Easily integrates with testing frameworks like [TestNG](#), [JUnit](#), and [Cucumber](#), enhancing flexibility and extensibility.

Importance of Testing in Selenium

Manual testing can be time-consuming and prone to human errors. Selenium Automation allows tests to be executed quickly and accurately, reducing the likelihood of human mistakes and ensuring consistent test results. Selenium allows developers and testers to automate the testing of web applications across different browsers and platforms.

1. **Language Support:** Selenium allows you to create test scripts in different languages like [Ruby](#), [Java](#), [PHP](#), Perl, [Python](#), [JavaScript](#), and [C#](#), among others.
2. **Browser Support:** Selenium enables you to test your website on different browsers such as [Google Chrome](#), [Mozilla Firefox](#), [Microsoft Edge](#), [Safari](#), [Internet Explorer \(IE\)](#), etc.
3. **Scalability:** Automated testing with Selenium can easily scale to cover a wide range of test cases, scenarios, and user interactions. This scalability ensures maximum test coverage of the application's functionality.
4. **Reusable Test Scripts:** Selenium allows testers to create reusable test scripts that can be used across different test cases and projects. This reusability saves time and effort in test script creation and maintenance.
5. **Parallel Testing:** Selenium supports parallel test execution, allowing multiple tests to run concurrently. This helps reduce the overall testing time, making the development process more efficient.
6. **Documentation and Reporting:** Selenium provides detailed test execution logs and reports, making it easier to track test results and identify areas that require attention.
7. **User Experience Testing:** Selenium can simulate user interactions and behavior, allowing testers to assess the user experience and ensure that the application is intuitive and user-friendly.
8. **Continuous Integration and Continuous Deployment (CI/CD):** Selenium can be integrated into CI/CD pipelines to automate the testing of each code change. This integration helps identify and address issues earlier in the development cycle, allowing for faster and more reliable releases.

Components of Selenium

Selenium Suite has 4 components namely:

1. Selenium IDE
2. Selenium RC
3. Selenium WebDriver
4. Selenium Grid

Selenium IDE

Selenium IDE is a Chrome and Firefox plugin. The primary use of a Selenium IDE is to record user interactions such as clicks, selections etc in the browser and plays them back as automated tests. It then generates the test script (of the automated tests) in programming languages like C#, Java, Python, and Ruby and Selenese (Selenium's own scripting language).

Selenium IDE helps in:

- Creating automated test scripts and validating them at speed
- Identifying and highlighting errors during the replay of interactions
- Cross Browser Testing

Selenium RC

Selenium RC was built to automate the testing of web applications by simulating user interactions across different browsers and platforms. It provided a way to browser automation remotely and execute test scripts written in various programming languages.

Selenium WebDriver

Selenium WebDriver is a powerful and enhanced version of Selenium RC which was developed to overcome the limitations of Selenium RC. WebDriver communicates with browsers directly with the help of browser-specific native methods, thereby completely eliminating the need of Selenium RC. WebDriver works closely with Selenium IDE and Selenium Grid resulting in reliable test execution at speed and scale.

Selenium Grid

Selenium Grid is a smart proxy server that allows QAs to run tests in parallel on multiple machines. This is done by routing commands to remote web browser instances, where one server acts as the hub. This hub routes test commands that are in JSON format to multiple registered Grid nodes.

Browsers Supported in Selenium WebDriver

Selenium supports these Browsers:

- Google Chrome
- Mozilla Firefox
- Safari
- Internet Explorer
- Microsoft Edge

Selenium WebDriver is a powerful browser automation tool designed to test web applications efficiently. Unlike Selenium RC, WebDriver directly interacts with browser elements, making it faster and more reliable. It supports multiple browsers, platforms, and programming languages, enabling comprehensive testing across different environments.

When to use Selenium WebDriver

Selenium WebDriver is primarily designed to test web applications.

- **Functional Testing:** Test the functionality of web applications by automating user interactions such as clicking buttons, filling out forms, navigating pages, and verifying expected outcomes.
- **Cross-Browser Testing:** Test web application for consistency across different browsers and browser versions (e.g., Chrome, Firefox, Edge, Safari)
- **Cross-Platform Testing:** Test the web application on different operating systems such as Windows, macOS, and Linux.
- **Regression Testing:** Test the existing functionality of the web app, whenever a new feature is introduced.
- **Data-Driven Testing:** Execute the same test script with different sets of input data to validate different scenarios.
- **Parallel Testing:** Selenium WebDriver can be integrated with tools like Selenium Grid to execute tests in parallel across multiple browsers and platforms, reducing testing time.
- **Complex User Flows:** Simulate complex user workflows or interactions that need to be tested repeatedly.

- **Integration with Continuous Integration (CI) Pipelines:** Selenium WebDriver can be seamlessly integrated into your [CI/CD pipeline](#), enabling automated testing with every code change and ensuring code quality.
- **UI/UX Testing:** Automate user interface (UI) testing, ensuring that the visual elements and layout of your application are consistent.
- **End-to-End Testing:** Selenium WebDriver can be used for end-to-end testing, where you simulate real user interactions across different parts of the application to ensure a seamless user experience.
- **Page Object Model (POM) Implementation:** If you adopt the Page Object Model (POM) design pattern, WebDriver can be used effectively to encapsulate the interactions and elements of different web pages, leading to more maintainable and modular test scripts.
- **Complex Scenarios:** When dealing with complex scenarios such as [handling alerts](#), [pop-ups](#), [iframes](#), and dynamic content, Selenium WebDriver provides the flexibility to address these challenges.
- **Performance Testing:** While not its primary use, WebDriver can be used to simulate user load and interactions for basic performance testing.

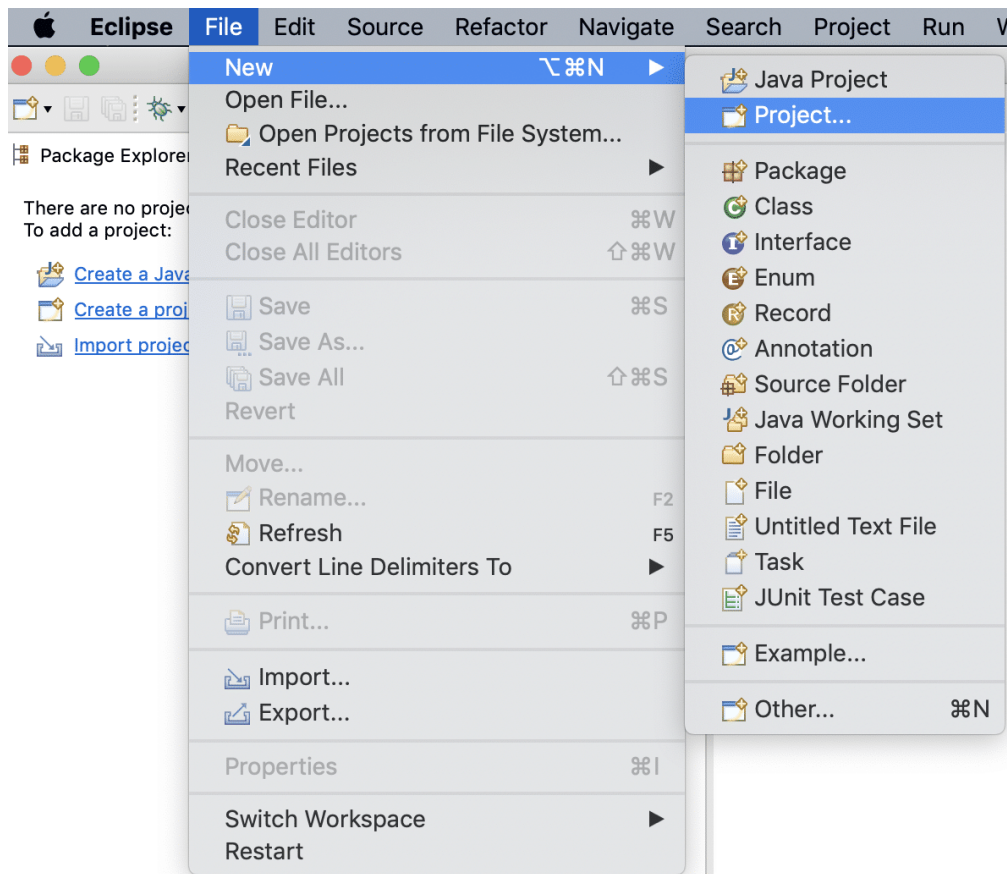
Configure Selenium WebDriver with Eclipse

[Eclipse](#) is one of the most used IDE by the java developers. In a project, all of these IDEs provide their ways to integrate and use functionalities of those libraries in your project. All the Java libraries are bundled and provided as JARS (Java Archives), which can be included in a project and then the project can invoke the needed functions of various classes bundled in those jar files. Similarly, Selenium WebDriver for Java also provides a set of jar files, which we need to include in the project to access and use the functionalities provided by Selenium WebDriver.

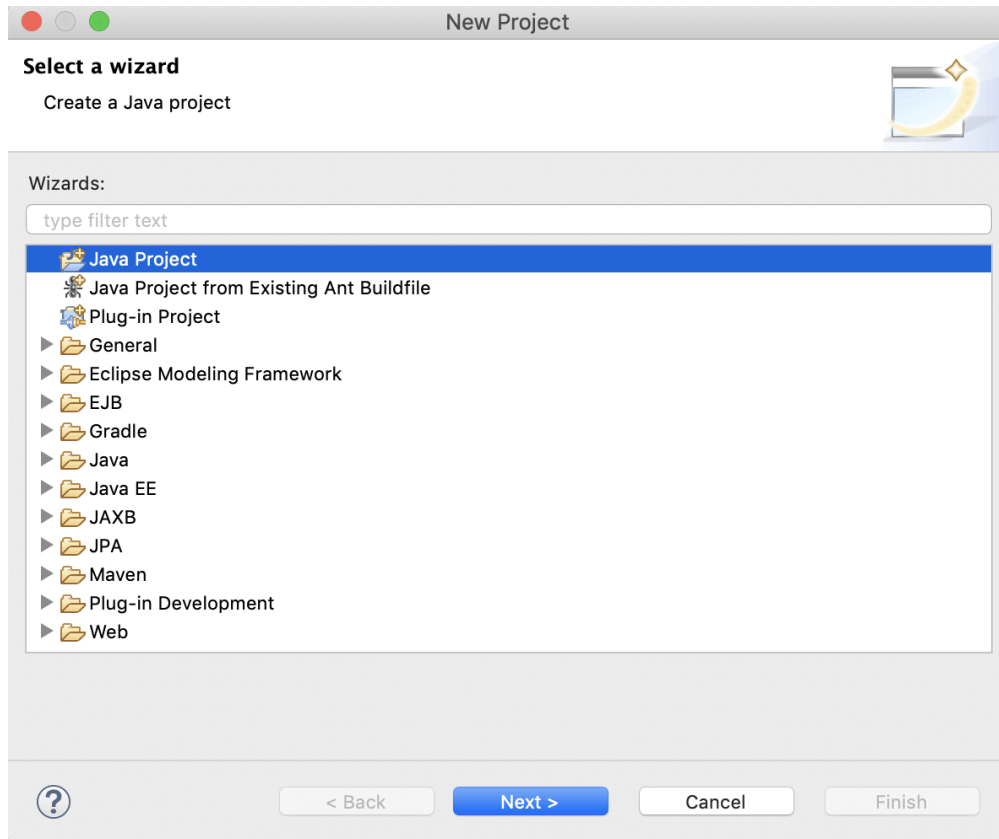
- How to launch Eclipse and create a workspace?
 - Create a new project in Eclipse
 - Create a new package under the project
 - Create a new class under the package
 - Add Selenium WebDriver Jars to the project

Create a new project in Eclipse?

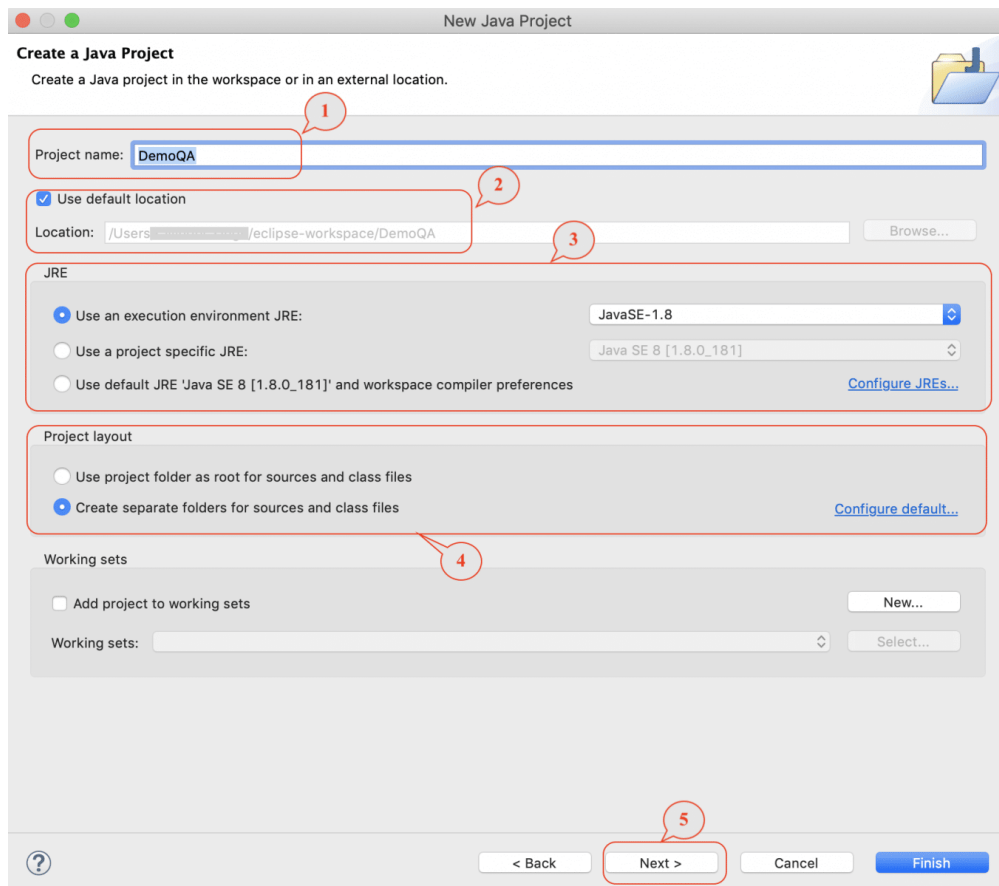
1. Navigate to **File > New > Project**. Alternatively, you can directly select the **Java Project** option under the **New** menu.



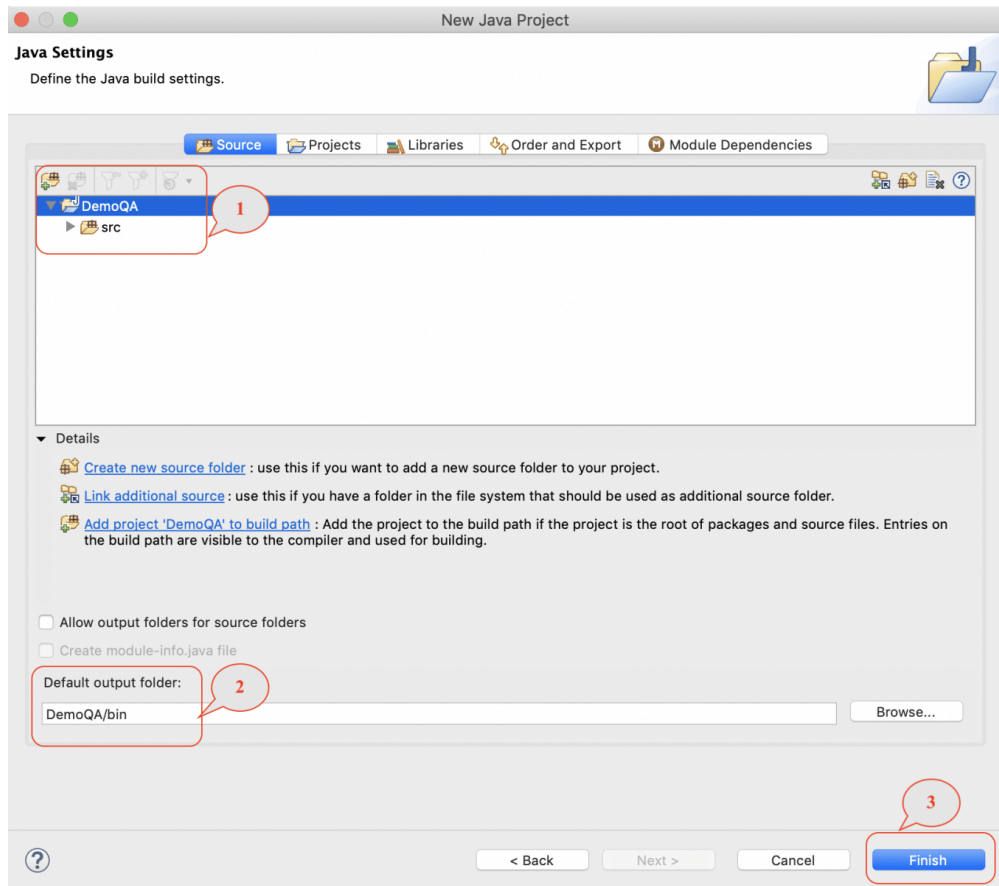
2. It will open the "New Project " wizard. From the **New Project** dialogue box, select **Java Project** and click on **Next**.



3. On the **"Create a Java Project"** wizard, specify a name to the Java project and select other options as shown below:



After completing all the configurations as mentioned above, either you can click on the **"Finish"** button to close the dialogue box or click on the **"Next"** button, as heightened by **marker 5**, which will open the dialogue box showing the folders for the project, sources, and classes as shown below:



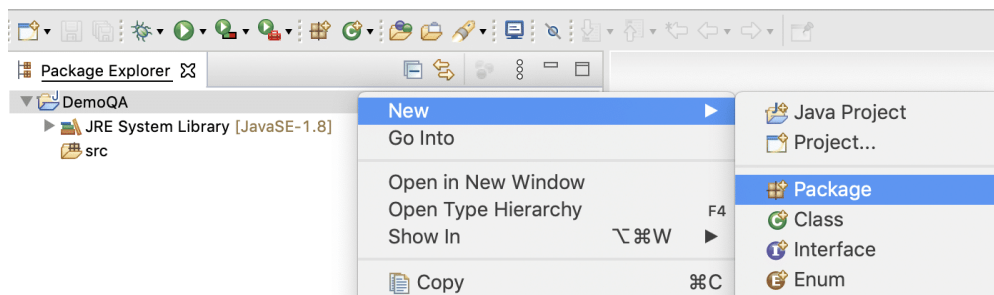
- As we can see in the above screenshot, Eclipse created a folder "**src**" (highlighted as 1). It will store all the source files and created a folder "**bin**" (highlighted as 2), which will store all the compiled class files. Click on the "**Finish**" button (highlighted as 3) to close the dialogue box.
- 4. Clicking on the "**Finish**" button will show a pop-up to select the Java perspective, which is more around aligning the different components of a Java project in the left Explorer box of Eclipse.

Once we finish creating a Java project in Eclipse, the next step is to create a [package](#) under the Project.

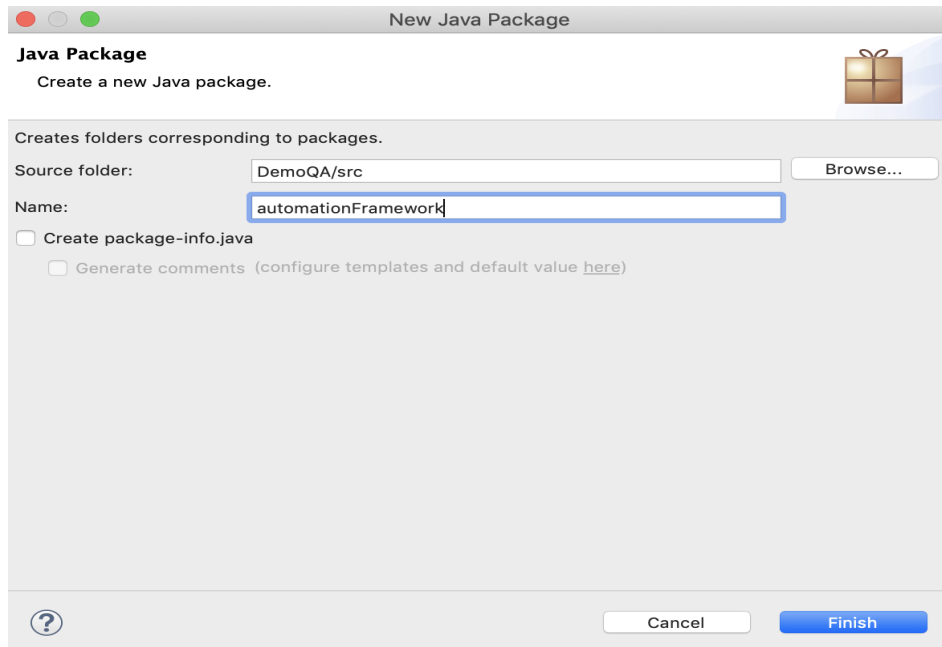
Create a new package under the project

To create a new package under the newly created project "**DemoQA**", follow the steps as mentioned below:

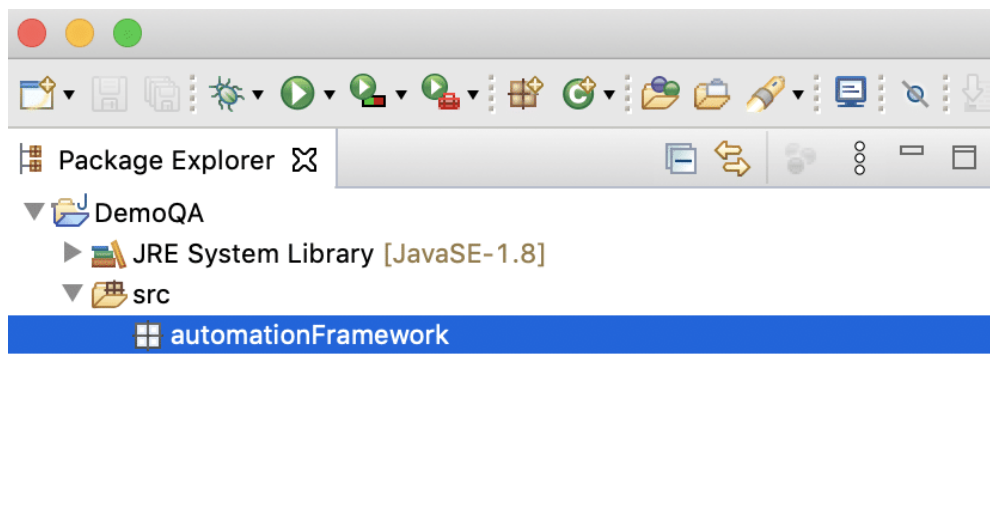
1. Right-click on your project name '*DemoQA*' and navigate to **New > Package**.



2. Give a name to your package, say, 'automationFramework' and click on the **Finish** button.



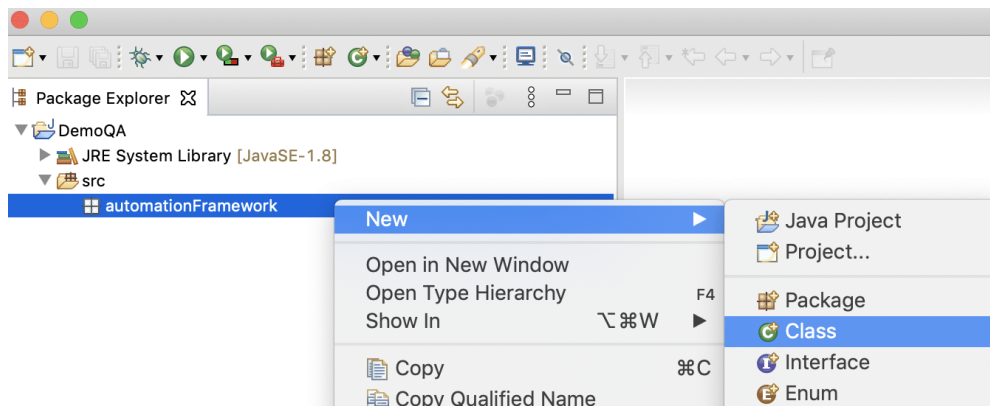
- Once the package creation is successful, it will reflect under the **src** folder of the project in the **Eclipse Project Explorer** windows, as shown below:



Once you finish creating a package under the project, the next step is to create a **Java class** under the package.

Create a new class under the package

- Right-click on the package 'automationFramework' and navigate to **New > Class**.



- Once you click on the **"Class"** option, You will see a **New Java Class** wizard as shown below:

New Java Class

Create a new Java class.

1. Source folder: DemoQA/src [Browse...]

Package: automationFramework [Browse...]

Enclosing type: [Browse...]

2. Name: FirstTestCase 3.

4. Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static

Superclass: java.lang.Object [Browse...]

5. Interfaces: [Add... Remove]

Which method stubs would you like to create?

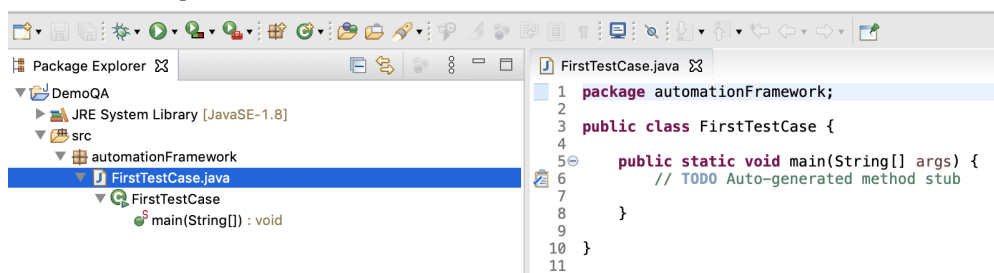
- ☒ public static void main(String[] args)
- ☐ Constructors from superclass
- ☐ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))

☐ Generate comments

6. [Cancel] [Finish]

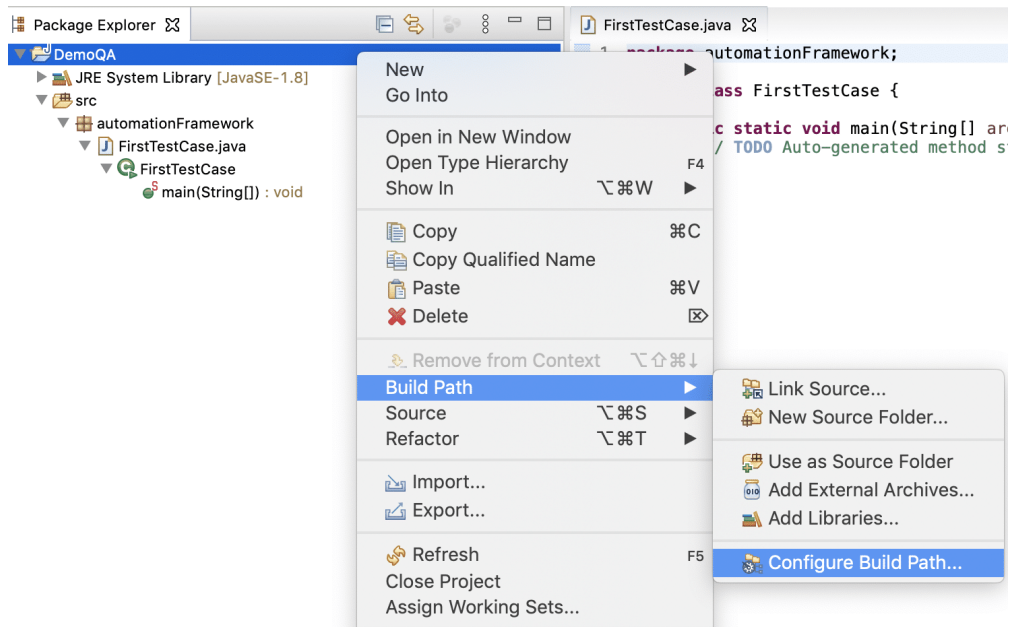
3. Once we create the Java class successfully. It will reflect under the "Project Explorer" window of Eclipse as shown below:



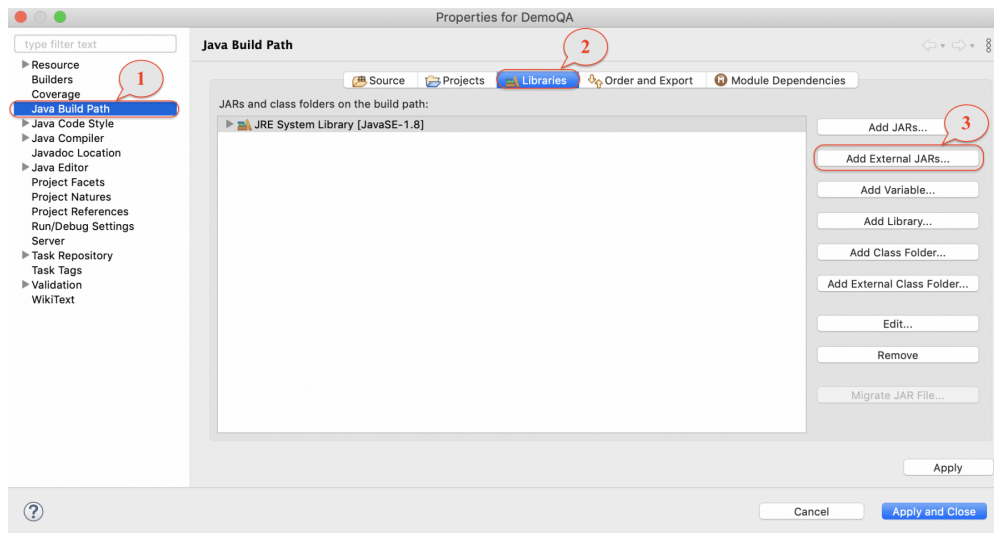
How to add Selenium WebDriver Jars to the project?

To include any of the Java libraries in the Eclipse projects, the same need to have in the ***Build Path of Eclipse**. Following the same, the Selenium WebDriver libraries also need to be included in the "Build Path" of Eclipse. As we have already [downloaded the Selenium WebDriver](#), We can include those **Selenium WebDriver Jar files** into the **Eclipse project** by following the steps as mentioned below:

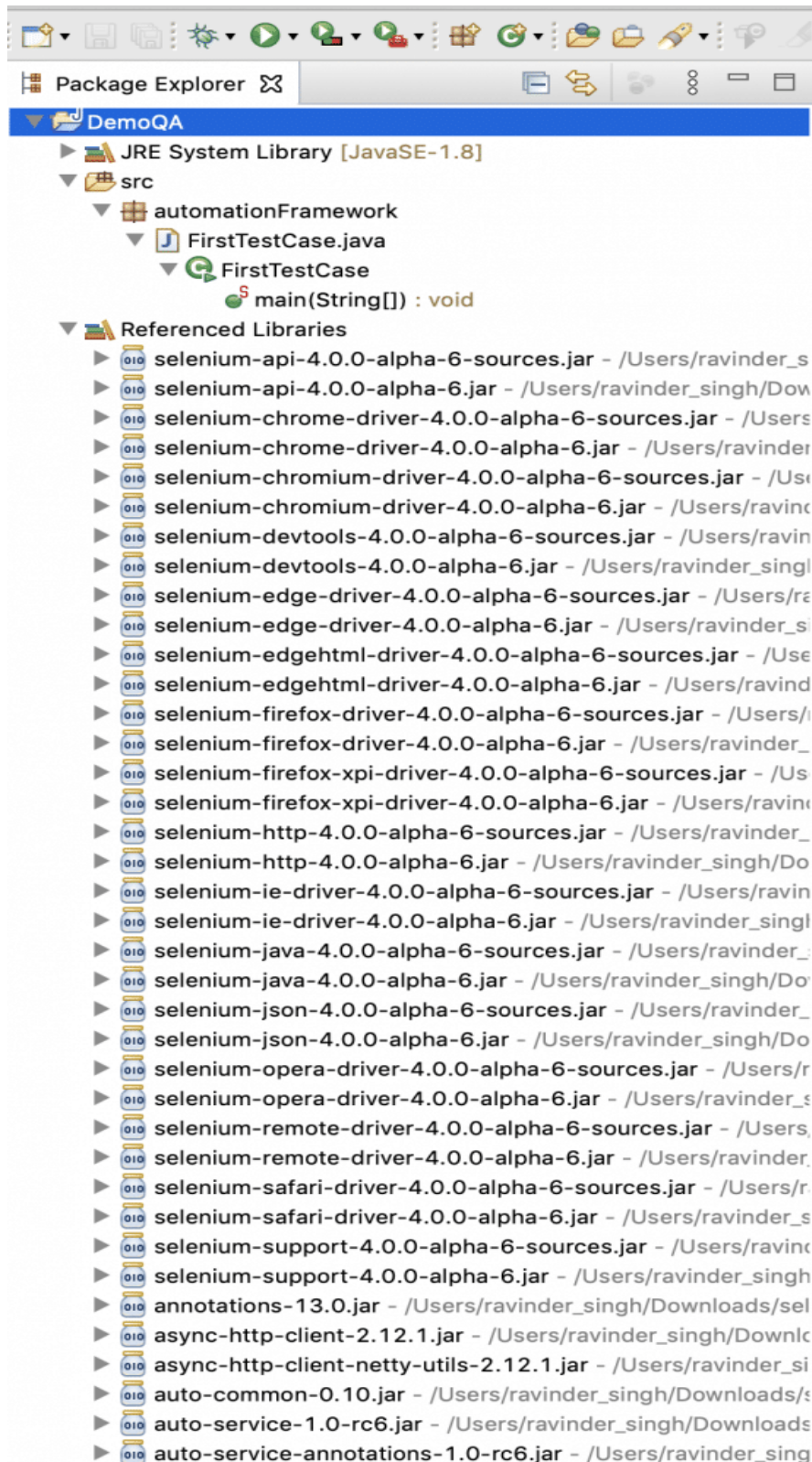
1. Firstly, either Right-click on the project 'DemoQA' > **Build Path** > **Configure Build Path**



2. Secondly, as "Selenium WebDriver" is an external library for the project, so we need to add the "Selenium WebDriver" dependencies as "External JARs".



- First, select the "Java Build Path" (as shown by marker 1) in the left panel, if it is not auto-selected.
 - After that, click on the "Libraries" tab, as shown by marker 2.
 - Finally, click on the "Add External JARs" button, as shown by marker 3.
3. Thirdly, browse to the folder where you have extracted all the Selenium WebDriver jar files.
 4. Lastly, once you finish adding all the "JAR" files, click on the button "Apply and Close". It will show all the JAR files in the "Project Explorer" window under the "Referenced Libraries" folder as shown below:



The configuration of the **Selenium WebDriver jars** is now complete. Conclusively, you are all set to write your **first test script using Selenium WebDriver in Eclipse**.

TestNG

TestNG is a testing framework inspired from JUnit and NUnit but introducing some new functionality that makes it more powerful and easier to use. It is an open-source automated testing framework; where NG of TestNG means Next Generation. TestNG is similar to JUnit but it is much more powerful than JUnit. It is designed to be better than JUnit, especially when testing integrated classes.

TestNG eliminates most of the limitations of the older framework and gives the developer the ability to write more flexible and powerful tests with help of easy annotations, grouping, sequencing & parametrizing.

Benefits of TestNG

1. It gives the ability to produce HTML Reports of execution

2. Annotations made testers life easy
3. Test cases can be Grouped & Prioritized more easily
4. Parallel testing is possible
5. Generates Logs
6. Data Parameterization is possible

What is a Selenium WebElement?

A **WebElement** in Selenium represents an HTML element on a web page, allowing interaction (click, send keys, retrieve text) through methods like **click()**, **sendKeys()**, and **getText()**. WebElements provide methods to interact with those elements, retrieve information, and verify element properties.

The WebElement interface in Selenium enables interaction with both visible and hidden elements on a web page. WebElement command returns either null/void or the element found.

Syntax of Selenium WebElement

<start tag> content </end tag>

- HTML elements can contain other elements. Every HTML document carries such HTML elements.
- WebElement Selenium WebDriver methods apply to almost all DOM elements on a web page.
- Each WebElement is represented in [Selenium](#) via the WebElement interface – which is used by Selenium to interact with visible and invisible elements on the web page.

Example of a WebElement in Selenium command:

```
WebElement element = driver.findElement(By.id("UserName"));
```

List of Selenium WebElement Commands

1. **sendKeys()** – Type text into input fields.
2. **isDisplayed()** – Check if the element is visible on the page
3. **isSelected()** – Check if the element is selected
4. **submit()** – Submits a form or triggers a submit action on a form element
5. **isEnabled()** – Check if the element is enabled
6. **getLocation()** – Returns values of the x and y coordinates of the WebElement relative to the top-left corner of the viewport.
7. **clear()** – Clear text from an input field
8. **getText()** – Get the visible text inside the WebElement.
9. **getTagName()** – Returns the tag name of the WebElement as a string. The tag name is the type of the HTML element.
10. **getCssValue()** – Get the value of a specified CSS property of a WebElement.
11. **getAttribute()** – Get the value of a specific attribute of the element
12. **click()** – Click on the WebElement
13. **getSize()** – Returns values of height and width of the WebElement.

What is testng.xml file and its purpose?

TestNG is a popular testing framework in Java that provides several features for writing test cases. One of the key components of TestNG is the 'testng.xml' file, which allows for easy configuration and control over test execution.

The 'testng.xml' file is a configuration file used by the [TestNG](#) testing framework in Java. It allows you to define and organize your test suite, specify which test classes to run, and configure various aspects of your test execution. This XML file provides a way to manage complex test scenarios and control the test execution in a more structured way.

It allows for:

- Grouping test cases into test suites and test groups.
- Specifying the order of execution.
- Controlling parallel execution and setting thread count.
- Filtering which test methods to run.

What is the Purpose of the testng.xml File?

1. **Centralized Configuration:** Instead of hardcoding configurations into individual test classes, the testng.xml file allows you to configure your entire test suite in a single place. This makes it easier to adjust test settings, change the execution order, or add/remove tests without modifying the test code itself.
2. **Modularity:** The file lets you create multiple test groups, suites, and sub-suites, which enables

modular testing. This is useful for organizing tests based on functionality, teams, or release cycles.

3. **Control Over Execution:** You can use the testng.xml to set the execution order of test cases, manage parallel execution, and even specify conditions under which certain tests should or shouldn't run (e.g., based on environment, priority, or dependency).
4. **Filter and Fine-tune Tests:** By specifying only certain methods or classes to run, you can selectively execute parts of your test suite. This is beneficial when you're testing specific features or modules.
5. **Ease of Maintenance:** The file promotes better maintenance of your test suite, as you can modify test configurations without altering the test scripts. This separation of concerns makes scaling and updating test suites more manageable.

Code Example

Step 1. Running Tests with testng.xml

```
package com.example.tests;

import org.testng.annotations.Test;

public class TestClass1 {
    @Test
    public void testMethod1() {
        System.out.println("TestClass1 - testMethod1 executed");
    }
}

public class TestClass2 {
    @Test
    public void testMethod2() {
        System.out.println("TestClass2 - testMethod2 executed");
    }
}
```

TEST REPORTING

Test reporting is a process in software testing that involves gathering, analyzing, and presenting essential test results and statistics to stakeholders. Additionally, a Test Report is a detailed document that contains a summary of the test, the process involved and the final test results. A Test Report is a summary of all the tests that were done, what was intended to be achieved with them, and what were the testing results once the tests were complete. It aims to summarize the results of the entire testing process formally and shares a comprehensive view of the testing status overall.

Importance of Test Reporting

Test reporting is crucial for software development from an ROI perspective. It helps:

- Maintain Cost-effectiveness
- Ensure release readiness
- Improve User Churn Rate
- Better Visibility and Control

Maintain Cost-Effectiveness

A good Test Report helps identify critical bugs very early in the cycle. It helps in the Root Cause Analysis, empowering the team with better debugging capabilities. This saves a lot of time and cost as the testers do not need to manually run the tests and report issues. Detecting a bug faster is always crucial as it can **save up to 85% of the cost** that would occur if they are caught later in the cycle.

Ensure Release Readiness

Getting a clear sense of release readiness with test coverage assessment, defect root cause analysis, and requirements validation by Test Report helps deliver high-quality software. Test Report validates if the requirements are successfully met. It aids in decision-making by presenting the current status of the software under test. It helps identify defects and issues, classifying them based on severity, which makes it easier to decide the release readiness.

Improving User Churn Rate

Software industry is highly competitive, and users expect a seamless bug-free user experience to continue using an application.

Test reports have detailed information about the number of tests executed, test scenarios, no of the tests that were passed/failed, known defects, if any and much more. These details provide valuable insights about the overall product health.

Test Report contributes to **continuous improvement** in the testing process. Teams can analyze the report to identify areas for process enhancement and optimization. This can ensure a bug-free software release for the users, improving the overall user churn rate with a better experience.

Better Visibility and Control

Test Report allows project stakeholders to track progress, identify bottlenecks, and make informed decisions, especially for larger projects. It enables project managers to have better control over the testing activities.

Test reports provide the necessary information and valuable insights to stakeholders about the product quality, help them keep track of the product and decide whether it is ready for release. It also helps them in measuring whether it is aligning with the business requirements.

Types of Test Reports

- **Test Summary Report**: It provides a high-level overview of the testing activities conducted during a specific phase or cycle of the project. It includes metrics such as the number of test cases executed, passed, failed, and any outstanding defects. The summary report helps stakeholders understand the overall status of the testing effort.
- **Test Execution Report (TER)**: It provides detailed information about the execution of test cases, including the test case ID, description, status (pass/fail), and any comments or observations from the tester. TER allows stakeholders to track the progress of testing and identify areas that may require further attention.
- **Defect Report/Bug Report**: It documents any defects or issues identified during testing. It typically includes information such as the defect ID, description, severity, priority, steps to reproduce, and status. The defect report helps prioritize and manage the resolution of issues found during testing.
- **Traceability Matrix**: A traceability matrix maps requirements to test cases, ensuring that each requirement has been adequately tested. It helps verify that all project requirements have been addressed and provides a clear understanding of the test coverage.
- **Performance Test Report**: This report presents the results of performance testing, including metrics such as response time, throughput, resource utilization, and scalability.
- **Security Test Report**: For applications handling sensitive data or operating in security-critical environments, a security test report documents the findings of security testing activities. It identifies vulnerabilities, potential threats, and recommendations for improving the security posture of the software.
- **Regression Test Report**: When conducting regression testing, a report may be generated to summarize the impact of changes on existing functionality. It includes information about the test cases executed, any regressions detected, and the overall stability of the system after changes.
- **Compliance Test Report**: In regulated industries such as healthcare or finance, compliance test reports ensure that software meets relevant regulatory requirements and standards. These reports document adherence to specific guidelines and may be required for certification or approval.
- **User Acceptance Test (UAT) Report**: This report documents the results of user acceptance testing, where end-users validate whether the software meets their requirements and

expectations. It includes feedback from users, any issues encountered, and the overall readiness of the software for deployment.

Components of Test Report

1. **Test Objective** includes the purpose of the testing and clear requirements from the overall testing cycle.
2. **Areas Covered** include the overall regions and the functionalities of the product that were covered in testing. It need not capture every test scenario in detail but should cover all the areas at a high level.
3. **Areas Not Covered** capture the product areas that were not covered in testing. Any areas not tested are clearly highlighted, setting expectations accordingly. It also specifies the reasons for the areas not covered and the risks involved.
4. **Testing Approach** indicates the systematic method used to plan, design, execute, and evaluate tests based on objectives, constraints, and requirements.
5. **Defect Report** captures details about the defects identified, its root cause analysis, and management. Though it usually gets captured already during the testing process. However, including it in the test summary report gives additional information and insights about the defects.
6. **Platform Details** include details of the platform and environment used during testing. This helps in troubleshooting if any defects are identified. Also, helps in analyzing coverage, risks, and robustness of the testing.
7. **Overall Summary** includes commentary on the application's overall status under test. It informs the stakeholders about the critical issues discovered and its status (open/closed). It helps understand the overall readiness of shipping the product to the customer.

Characteristics of a Good Test Report

Some of the core characteristics of a good test report are:

- It captures the purpose of the document clearly.
- It gives a high-level overview of the product's functionality that is being tested.
- It captures the test scope, highlighting:
 - Modules in scope
 - Modules out of scope
 - Module that was in scope but couldn't be tested due to any underlying constraint or dependency.
- It captures different testing metrics for all kinds of tests that were performed, majorly
 - No. of test cases planned
 - No. of test cases executed
 - No. of test cases passed
 - No. of test cases failed
- It includes the different types of Testing that were performed, such as Unit Testing, Smoke Testing, Sanity Testing, Regression Testing, and Performance Testing
- It specifies the Test Environment, Tools and Frameworks used for testing.
- It clearly defines the Exit Criteria, which, once successfully met, the Testing team gives a go-ahead for the application to Go Live.